

Vector Clocks

A Simple Guide for System Design & Distributed Systems

1. The Problem: Why do we need Vector Clocks?

In a single computer, keeping track of time is easy. But in a **Distributed System** (many computers working together), tracking the order of events is surprisingly hard.

- **Physical Clocks Drift:** You cannot trust wall-clock time (NTP) across different servers because one server's clock might be slightly faster or slower than another's.
- **Lamport Clocks:** An earlier solution. They give events a number (e.g., Event A = 1, Event B = 2). If A caused B, A will have a lower number. However, if you see Event A=1 and Event C=5, Lamport Clocks *cannot* tell you if A caused C, or if they were completely independent.

The Solution: Vector Clocks solve this. They don't just tell us the order of events; they explicitly capture **Causality** (did Event A cause Event B, or did they happen independently?).

2. What is a Vector Clock?

Instead of a single number, a Vector Clock is an **array (a list) of numbers**. If your system has 3 nodes (Node A, Node B, Node C), the vector clock will have 3 slots: **[A, B, C]**.

Everyone starts at zero: **[0, 0, 0]**.

3. The Golden Rules of Vector Clocks

Nodes update their clocks based on three simple rules:

Rule 1: Internal Events

Before a node does anything (reads a database, updates a record, sends a message), it increments its own counter in its own vector by 1.

Example: Node A does some work. Its clock goes from **[0, 0, 0]** to **[1, 0, 0]**.

Rule 2: Sending Messages

When a node sends a message to another node, it attaches its current Vector Clock to the message.

Example: Node A sends a message to Node B. The message carries the clock **[1, 0, 0]**.

Rule 3: Receiving Messages

When a node receives a message, it does two things:

1. It increments its **own** counter by 1.
2. For every other counter, it takes the **maximum** value between its own vector and the received vector.

Example: Node B receives the message from A. Node B was at **[0, 0, 0]**. First, B updates its own counter: **[0, 1, 0]**. Then it takes the max of **[0, 1, 0]** and A's message **[1, 0, 0]**. New clock for Node B: **[1, 1, 0]**.

4. How to Read Causality (Comparing Clocks)

To know if Event V1 caused Event V2, we compare their vectors. Let's look at the three possible relationships:

Relationship	How to check the Math	Example
Happened Before (V1 caused V2)	Every number in V1 is $\leq$ the corresponding number in V2, AND at least one number is strictly $<$.	V1: [1, 0, 0] V2: [1, 1, 0] <i>V1 happened before V2.</i>
Happened After (V2 caused V1)	Every number in V1 is $\geq$ the corresponding number in V2.	V1: [2, 1, 1] V2: [1, 0, 1] <i>V1 happened after V2.</i>
Concurrent (Independent / Conflict)	Neither is strictly greater or lesser. One counter is higher in V1, but another is higher in V2.	V1: [1, 2, 0] V2: [2, 0, 0] <i>These happened independently! A conflict!</i>

5. Pros and Cons

Pros (Why use them?)	Cons (The Trade-offs)
• Accurate Causality: It is mathematically proven to detect if events happened sequentially or concurrently. • Detects Conflicts: In a database, if two writes are concurrent, the database knows it must ask the client to resolve the conflict.	• Overhead: The size of the vector grows with the number of nodes. If you have 10,000 nodes, every message carries an array of 10,000 numbers. • Complexity: More complex to implement than a simple timestamp.



Real-World Example: Amazon DynamoDB

Amazon's original Dynamo database paper famously used Vector Clocks to handle the "Shopping Cart" problem. If you add "Shoes" on your phone (hits Node A) and add a "Hat" on your laptop (hits Node B) at the same time, the system might have a network partition. Vector Clocks detect that these two additions are *Concurrent*. Instead of blindly overwriting one cart with the other, Dynamo keeps both versions and asks the application to merge them (resulting in a cart with both Shoes and Hat).